

Schemas, Constraints, and Integrity Testing

How table design and database rules keep support requests consistent.

Counting a subset within each category

```
SELECT category,  
       count(*) AS total_tickets,  
       count(*) FILTER (  
         WHERE status = 'resolved'  
       ) AS resolved_tickets  
FROM metro_support.tickets  
GROUP BY category  
ORDER BY category;
```

Category	Total	Resolved
parks	2	0
sanitation	3	1
streetlight	3	1
transportation	2	0
water	2	2

FILTER affects this one aggregate. It leaves the category and its total count in the result.

Totals: 12 tickets, 4 resolved. The lab changes resolved to its definition of active.

Schema as a design and a namespace

Meaning	What it describes	Example
Database design	Tables, columns, relationships, and rules	A ticket requires an existing requester
PostgreSQL namespace	A named container inside one database	metro_support contains tickets

```
SELECT ticket_id, status  
FROM metro_support.tickets;
```

metro_support = schema name
tickets = table name

A new editor tab can have a different search_path. The qualified name states the intended table.

One home for a current contact fact

Hypothetical design: copy the current email into every ticket

Ticket	Requester	Current email
1001	101	maya.chen@example.test
1005	101	maya.chen@example.test
1009	101	maya.chen@example.test

User	Current email
101	maya.chen@example.test

Course design: the email lives in users.
Each ticket stores requester_id.

requester_id → current email

Changing one copied email can leave conflicting current facts. A historical snapshot is a different fact.

Data types express the intended value

Need	PostgreSQL type	Why this choice
Whole-number identifier	integer	Equality and relationships, not text spelling
Subject or status text	text	A CHECK can add an allowed vocabulary
An instant in time	timestampz	Compare instants even across display time zones
A calendar date	date	No time of day is implied
Exact decimal amount	numeric(p,s)	Avoid binary approximation when exact decimals matter

A type alone does not establish the business rule. A postal code is usually text, even when it contains digits.

Keys, missing values, and references

Row identity

ticket_id is unique and required.
users.email also has a UNIQUE rule.

One identifier per ticket

Value domain

requester_id is required. assignee_id
may be NULL while work is
unassigned.

Missing can be a valid state

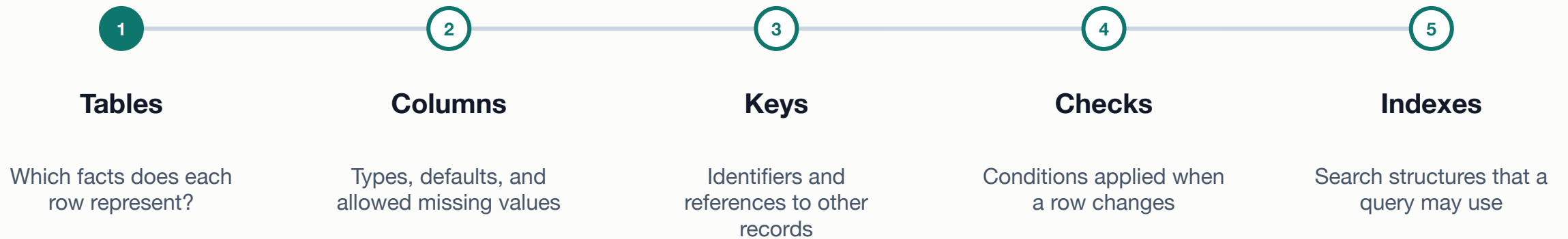
Relationship

A non-null requester_id or
assignee_id must match an existing
user_id.

Existence does not prove role

Reading the current database structure

Metadata describes the objects and rules currently installed in this database.



A query about columns

```
SELECT column_name, data_type,  
       is_nullable, column_default  
FROM information_schema.columns  
WHERE table_schema = 'metro_support'  
      AND table_name = 'users'  
ORDER BY ordinal_position;
```

Reading this result

- Six result rows describe six columns. The table contains eight users.
- is_nullable = NO means the column requires a value.
- Change 'users' to 'tickets' to inspect the lab table.

The saved constraint definitions

```
SELECT conname, contype,  
       pg_get_constraintdef(oid) AS definition  
FROM pg_constraint  
WHERE conrelid =  
       'metro_support.tickets'::regclass  
ORDER BY conname;
```

Code	Meaning	Baseline example
p	Primary key	ticket_id
f	Foreign key	requester_id references users(user_id)
c	Check	closed_at is absent or not before opened_at

PostgreSQL 18 also lists n = NOT NULL. In every version, inspect is_nullable in column metadata.

Constraints and indexes

Constraint

- The primary key rejects duplicate ticket identifiers.
- The priority CHECK rejects values outside its list.
- A foreign key requires a referenced user to exist.

Index

- PostgreSQL creates a unique index for a primary key.
- Other indexes can support filtering, joins, or sorting.
- Each index uses space and adds maintenance work.

Catalog query for the lab

```
SELECT indexname, indexdef FROM pg_indexes  
WHERE schemaname = 'metro_support' AND tablename = 'tickets';
```

Lab 1: Inspect before changing

Individual work using the complete baseline: 8 users, 12 tickets, and 21 events.

- Count all tickets and active tickets within each category.
- Inspect ticket columns, constraints, and indexes with the supplied queries.
- Explain the requester rule, optional assignee, and one missing protection.

Brightspace submission: week_03_schema_xray.sql

Allowed values and rejected changes

Today we add a priority rule, test both outcomes, and adapt it for status.

A misspelled status can hide a request

A row can contain 'IN PROGRESS' while a report searches for 'in_progress'.

STORED VALUE

A plain text column can accept either spelling.

REPORT FILTER

The two strings do not match under the course database settings.

DATABASE RULE

A CHECK can limit status to the approved vocabulary.

Examples of database integrity rules

Rule	Course example	Change the rule rejects
PRIMARY KEY	tickets.ticket_id	Give ticket 1004 the existing ID 1001
NOT NULL	tickets.requester_id	Set requester_id to NULL
FOREIGN KEY	requester_id refers to users	Set requester_id to nonexistent 999999
UNIQUE	users.email	Give user 102 the email already used by 101
CHECK	Allowed priority values	Set priority to extreme after installing the rule

Existing keys and missing-value rules are already in the baseline. The priority CHECK is our addition.

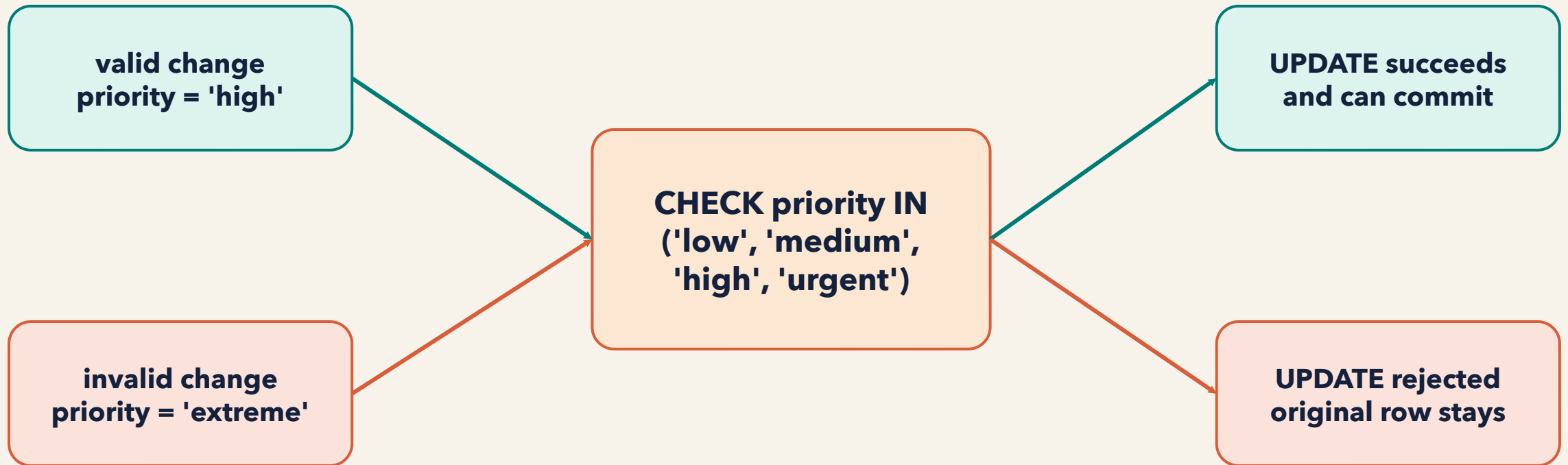
A complete priority constraint

```
SELECT priority, count(*)  
FROM metro_support.tickets  
GROUP BY priority ORDER BY priority;  
  
ALTER TABLE metro_support.tickets  
ADD CONSTRAINT tickets_priority_allowed  
CHECK (priority IN (  
    'low', 'medium', 'high', 'urgent'  
));
```

Before and after ADD

- Inspect existing values before PostgreSQL checks them against the new rule.
- The name tickets_priority_allowed identifies this particular rule.
- Add it once. On a repeat run, inspect the saved definition first.

How the priority rule handles an update



The database checks the proposed row before accepting the change.

Rejected and accepted priority changes

Run this expected failure separately

```
UPDATE metro_support.tickets  
SET priority = 'extreme'  
WHERE ticket_id = 1004;
```

Expected SQLSTATE: 23514
Rule: tickets_priority_allowed
Ticket 1004 stays low.

Then run this complete successful batch

```
BEGIN;  
UPDATE metro_support.tickets  
SET priority = 'high' WHERE ticket_id = 1004  
RETURNING ticket_id, priority;  
ROLLBACK;  
SELECT ticket_id, priority FROM metro_support.tickets  
WHERE ticket_id = 1004;
```

RETURNING shows high.
After ROLLBACK: low.

If the editor reports an aborted transaction, run ROLLBACK before retrying.

CHECK and missing values

Proposed priority	Allowed-list predicate	CHECK alone
'high'	TRUE	Accepts
'extreme'	FALSE	Rejects
NULL	UNKNOWN	Accepts

The baseline also declares priority NOT NULL. Together, the two rules require a value from the allowed list.

Recall: WHERE keeps only TRUE. CHECK rejects FALSE. The same three-valued logic has different uses.

Lab 2: Prevent misspelled statuses

Individual work: adapt the priority example to protect the status vocabulary.

- Keep the worked priority rule and write tickets_status_allowed.
- Run the two expected failures separately, then the valid change and rollback.
- Explain the protection and one limit in the same SQL file.

Brightspace submission: week_03_integrity_build.sql

What our schema now guarantees

Allowed words stay consistent across the form, an import, and a SQL script.

- Metadata shows the saved definition and its name.

- A valid change succeeds. An invalid change names the rejecting rule.

- The rule does not decide who may close a ticket. Week 6 studies access.